LoRA: Low-Rank Adaptation of Large Language Models

Rank Ablation Re-implementation Study

Veronica Aragon, Angela Martinez

CS5365–Deep Learning | University of Texas at El Paso | Spring 2026

1. Introduction

One of the biggest problems with modern language models is that they are expensive to fine-tune. A model like GPT-3 has 175 billion parameters, and updating all of them for a specific task requires a lot of memory and compute that most people simply don't have access to.

LoRA (Hu et al., 2022) proposes a simpler approach: instead of updating all the weights in the model, you freeze everything and only train two small matrices that get added on top. The idea is that the changes needed for a new task are actually low-dimensional, so you don't need to update billions of parameters to get good results.

For this project, we chose to re-implement the rank ablation experiment from the paper. This experiment tests how the size of those small matrices (controlled by a parameter called rank, or r) affects performance. We wanted to see if the paper's claim holds: that even a very small rank like r=1 works almost as well as r=64.

2. Chosen Result

We focused on reproducing Table 5 from the original paper, which shows how model performance changes when you vary the rank r across values: 1, 2, 4, 8, 16, and 64.

We picked this specific result because it is the core of what makes LoRA interesting. If a tiny rank works just as well as a large one, that means you can fine-tune a model with almost no extra parameters. That's the whole value of the method, so testing it directly made the most sense for our project.

Since we couldn't use GPT-3 like the original paper did, we ran our experiments on roberta-base and evaluated on two GLUE tasks: SST-2 (sentiment classification) and MRPC (paraphrase detection).

3. Methodology

We used the Hugging Face PEFT library to apply LoRA on top of roberta-base. LoRA was applied specifically to the query and value weight matrices in the attention layers, which is what the paper recommends. Everything else in the model was frozen.

Setup

- Base model: roberta-base (125M parameters)
- LoRA target layers: W_q and W_v (query and value projections)
- Tasks: SST-2 (accuracy) and MRPC (F1 score)
- Ranks tested: r ∈ {1, 2, 4, 8, 16, 64}

Training Settings

Parameter	Value
LoRA alpha	16
Dropout	0.1
Learning rate	3e-4
Batch size	32
Epochs	1

Max sequence length	128
Optimizer	AdamW

Differences from the original paper

The original paper used GPT-3, which we obviously could not replicate. We used roberta-base instead, which is much smaller but still a strong pretrained model. We also only trained for 1 epoch because of time and compute constraints — the full sweep of 12 runs (6 ranks x 2 tasks) took about 3 hours running on CPU through Google Colab. We think the trends are still valid even with these limitations.

4. Results & Analysis

Rank (r)	SST-2 Accuracy	MRPC F1	Trainable Params
1	0.9358	0.8179	~629K
2	0.9289	0.8167	~666K
4	0.9289	0.8167	~740K
8	0.9323	0.8197	~887K
16	0.9323	0.8221	~1.18M
64	0.9278	0.8159	~3.28M

The results were honestly surprising to us. We expected that higher ranks would perform noticeably better, but the gap between r=1 and r=64 is tiny. On SST-2, accuracy only varies by about 0.8% across all ranks (from 0.928 to 0.936). On MRPC, F1 score varies even less, about 0.6% (from 0.816 to 0.822).

What stands out is that r=1 actually got the highest SST-2 accuracy (0.9358) out of all ranks. This is a bit counterintuitive, but it lines up with what the paper argues: the useful information for adapting to a task lives in a very low-dimensional space, so adding more parameters doesn't always help and can sometimes hurt due to overfitting or noise.

Our numbers are lower than what the original paper reports, which makes sense given we used a smaller model and fewer training epochs. But the trend is the same: rank barely matters, and that's the point.

5. Reflections

This project taught us a lot, mostly through things going wrong. Getting the code to actually run took longer than expected because of version conflicts between PyTorch, Transformers, and PEFT. We went through several rounds of debugging just to get one training run to complete. If we did this again, we would pin the package versions at the start instead of letting pip install whatever it wanted.

We also learned that training on CPU is really slow. Each of the 12 runs took about 15-17 minutes, so the whole sweep took around 3 hours. In the future it would be worth setting up the environment to properly use the GPU from the beginning, which would cut that time significantly.

On the research side, we now have a much better intuition for what LoRA is actually doing. Before this project, the idea of low-rank adaptation seemed abstract. After seeing the results, it clicked: the model really does only need a tiny update to learn a new task, and that update can be represented with very few parameters.

If we had more time, we would try running more epochs to see if the rank differences become more noticeable with longer training, and test on a larger model like GPT-2 to see if the same pattern holds.

6. References

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. https://arxiv.org/abs/2106.09685

Wang, A., et al. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. EMNLP 2018.

Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.

Hugging Face PEFT Library. https://github.com/huggingface/peft